

Lab 2 Assignment

IIT Bhilai — Artificial Intelligence Lab Assignment (Do not write a code from scratch)

Instructions

This assignment builds directly on the A* Search problem that was discussed in the class (the Romania-map). For all three tasks below:

- Reuse the exact same functions and overall code flow discussed in the tutorial — `build_graph()`, `heuristic()`, `draw_graph()`, and `astar_search()` are all written the same way (explicit loops, a priority queue via `heapq`, and a visited set).
- Only change the specific line(s) or data mentioned in each task. Do not rewrite the algorithms from scratch, and do not introduce new libraries.
- Use the node coordinates and the map given below as your graph data for all three tasks. • Show your code, your printed output (path + cost), and a short written explanation for every task.

Romania Map — Node Coordinates

These are the (x, y) positions to use for plotting each city with `draw_graph()`. Build your edges and heuristic (straight-line distance to Bucharest) directly from the map given below.

Node (City)	X	Y
Oradea	131	571
Zerind	108	531
Arad	91	492
Timisoara	94	410
Lugoj	165	379
Mehadia	168	339
Dobreta	165	299
Sibiu	207	457
Fagaras	305	449
RimnicuVilcea	233	410
Pitesti	320	368

Craiova	253	288
Bucharest	400	327
Giurgiu	375	270

Urziceni 456 350
 Hirsova 534 350
 Eforie 562 293
 Vaslui 509 444
 Iasi 473 506
 Neamt 406 537

Romania Map

Road distances (edge weights, in km) and straight-line distances to Bucharest (heuristic values) are both given directly on this map — use these values as your edges and h_table.



Task 1 — Draw the Graph and Cross-Check Arad → Bucharest

Using the same `build_graph()`, `heuristic()`, and `draw_graph()` functions from the tutorial, construct the Romania graph from the map above (edges + weights + heuristic values) and draw it.

Then run `astar_search()` from Arad to Bucharest using the given heuristic table. Print the path and the total cost.

Cross-check: the known shortest distance from Arad to Bucharest is 418 km, via Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest. Your output should match this exactly and also try with a few other states.

Task 2 — Admissible vs Inadmissible Heuristic

As discussed in class, A* Search can return either an optimal or a suboptimal path depending on the heuristic. Create a scenario on the **Romania graph** where A* returns a **suboptimal path** instead of the optimal path.

Hint: Remember the optimality condition of A* and the role of an **admissible heuristic**. Do not

modify the `astar_search()` function. Please write your justification in the last cell of Task 2.

Task 3 — Greedy Best-First Search (Complete vs Not Complete)

You now need to implement **Greedy Best-First Search** using the existing **A* Search algorithm** as the base.

First, modify A* so that it works as **Greedy Best-First Search** by using only the heuristic value $h(n)$ for selecting the next node.

After implementing Greedy Best-First Search, use the **Romania graph** to create and demonstrate **two scenarios**:

1. **Complete Scenario:** Choose any starting state **except Arad** and Bucharest as the goal. Show a case where Greedy Best-First Search successfully reaches the goal and returns a path.
2. **Incomplete Scenario:** Choose any starting state **except Arad** and Bucharest as the goal. Create a scenario where Greedy Best-First Search does not successfully reach the goal or fails to terminate under the given search conditions.

For both scenarios, report the starting state, goal state, returned result, and briefly explain why Greedy Best-First Search is **complete or incomplete** in that particular scenario.

Important: Do not completely rewrite the search algorithm. Use the existing A* implementation as the base and make only the necessary change to turn it into Greedy Best-First Search. Please write your justification in the last cell of Task 3.

End of Assignment